CHƯƠNG 4 · BÀI 4

4.4. Các giải pháp đồng bộ hóa

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Phân biệt hai nhóm giải pháp đồng bộ: busy waiting và sleep & wakeup.
- Hiểu và phân tích được biến khóa, kiểm tra luân phiên, Peterson, cấm ngắt, TSL.
- Sử dụng được semaphore, monitor và trao đổi thông điệp để đồng bộ tiến trình.

4.4.1 Giải pháp "busy waiting"

4.4.1.1 Sử dụng các biến cờ hiệu(simaphore)

Tiếp cận: các tiến trình chia sẻ một biến chung đóng vai trò "chốt cửa" (lock), biến này được khởi động là 0. Một tiến trình muốn vào miền găng trước tiên phải kiểm tra giá trị của biến lock. Nếu lock = 0, tiến trình đặt lại giá trị cho lock = 1 và đi vào miền găng. Nếu lock đang nhận giá trị 1, tiến trình phải chờ bên ngoài miền găng cho đến khi lock có giá trị 0. Như vậy giá trị 0 của lock mang ý nghĩa là không có tiến trình nào đang ở trong miền găng, và lock = 1 khi có một tiến trình đang ở trong miền găng.

```
while (TRUE) {
while (lock == 1); // wait
lock = 1;
critical-section ();
lock = 0;
Noncritical-section (); }
```

4.4.1.2 Sử dụng việc kiểm tra luân phiên

Tiếp cận: Đây là một giải pháp đề nghị cho hai tiến trình. Hai tiến trình này sử dụng chung biến turn (phản ánh phiên tiến trình nào được vào miền găng), được khởi động với giá trị 0. Nếu turn = 0, tiến trình A được vào miền găng. Nếu turn = 1, tiến trình A đi vào một vòng lặp chờ đến khi turn nhận giá trị 0. Khi tiến trình A rời khỏi miền găng, nó đặt giá trị turn về 1 để cho phép tiến trình B đi vào miền găng.

```
while (TRUE) {
while (turn != 0); // wait
critical-section ();
turn = 1;
Noncritical-section (); }
```

(a) Cấu trúc tiến trình A

```
while (TRUE) {
while (turn != 1); // wait
critical-section ();
turn = 0;
Noncritical-section (); }
```

4.4.1.3 Giải pháp của Peterson

Tiếp cận: Petson đưa ra một giải pháp kết hợp ý tưởng của cả hai giải pháp kể trên. Các tiến trình chia sẻ hai biến chung:

```
int turn; // đến phiên ai
int interesse[2]; // khởi động là FALSE
```

Nếu interesse[i] = TRUE có nghĩa là tiến trình Pi muốn vào miền găng. Khởi đầu, interesse[0] = interesse[1] = FALSE và giá trị của est được khởi động là 0 hay 1. Để có thể vào được miền găng, trước tiên tiến trình Pi đặt giá trị interesse[i]=TRUE (xác định rằng tiến trình muốn vào miền găng), sau đó đặt turn=j (đề nghị thử tiến trình khác vào miền găng). Nếu tiến trình Pj không quan tâm đến việc vào miền găng (interesse[j]=FALSE), thì Pi có thể vào miền găng, nếu không, Pi phải chờ đến khi interesse[j]=FALSE. Khi tiến trình Pi rời khỏi miền găng, nó đặt lại giá trị cho interesse[i]= FALSE.

```
while (TRUE) {
int j = 1-i; // j là tiến trình còn lại
interesse[i]= TRUE;
turn = j;
while (turn == j && interesse[j]==TRUE);
critical-section ();
interesse[i] = FALSE;
Noncritical-section (); }
```

4.4.1.4 Cấm ngắt

Là giải pháp phần cứng.

Tiếp cận: cho phép tiến trình cấm tất cả các ngắt trước khi vào miền găng, và phục hồi ngắt khi ra khỏi miền găng. Khi đó, ngắt đồng hồ cũng không xảy ra, do vậy hệ thống không thể tạm dừng hoạt động của tiến trình đang xử lý để cấp phát CPU cho tiến trình khác, nhờ đó tiến trình hiện hành yên tâm thao tác trên miền găng mà không sợ bị tiến trình nào khác tranh chấp.

4.4.1.5 Chỉ thị TSL (Test-and-Set)

Là giải pháp phần cứng.

Tiếp cận: đây là một giải pháp đòi hỏi sự trợ giúp của cơ chế phần cứng. Nhiều máy tính cung cấp một chỉ thị đặc biệt cho phép kiểm tra và cập nhật nội dung một vùng nhớ trong một thao tác không thể phân chia, gọi là chỉ thị Test-and- Set Lock (TSL) và được định nghĩa như sau:

Test-and-Setlock (boolean target)

```
{ Test-and-Setlock = target;
target = TRUE; }
```

Nếu có hai chỉ thị TSL xử lý đồng thời (trên hai bộ xử lý khác nhau), chúng sẽ được xử lý tuần tự. Có thể cài đặt giải pháp truy xuất độc quyền với TSL bằng cách sử dụng thêm một biến lock, được khởi gán là FALSE. Tiến trình phải kiểm tra giá trị của biến lock trước khi vào miền găng, nếu lock = FALSE, tiến trình có thể vào miền găng.

```
while (TRUE) {
while (Test-and-Setlock(lock));
critical-section ();
lock = FALSE;
Noncritical-section (); }
```

Tất cả các giải pháp trên đây đều phải thực hiện một vòng lặp để kiểm tra liệu nó có được phép vào miền găng, nếu điều kiện chưa cho phép, tiến trình phải chờ tiếp tục trong vòng lặp kiểm tra này. Các giải pháp buộc tiến trình phải liên tục kiểm tra điều kiện để phát hiện thời điểm thích hợp được vào miền găng như thế được gọi các giải pháp “busy waiting”. Lưu ý rằng việc kiểm tra như thế tiêu thụ rất nhiều thời gian sử dụng CPU, do vậy tiến trình đang chờ vẫn chiếm dụng CPU. Xu hướng giải quyết vấn đề đồng bộ hoá là nên tránh các giải pháp “busy waiting”.

4.4.2 Các giải pháp “SLEEP and WAKEUP”

Để loại bỏ các bất tiện của giải pháp “busy waiting”, chúng ta có thể tiếp cận theo hướng cho một tiến trình chưa đủ điều kiện vào miền găng chuyển sang trạng thái blocked, từ bỏ quyền sử dụng CPU. Để thực hiện điều này, cần phải sử dụng các thủ tục do hệ điều hành cung cấp để thay đổi trạng thái tiến trình. Hai thủ tục cơ bản SLEEP và WAKEUP thường được sử dụng để phục vụ mục đích này. SLEEP là một lời gọi hệ thống có tác dụng tạm dừng hoạt động của tiến trình (blocked) gọi nó và chờ đến khi được một tiến trình khác “đánh thức”. Lời gọi hệ thống WAKEUP nhận một tham số duy nhất: tiến trình sẽ được tái kích hoạt (đặt về trạng thái ready).

Ý tưởng sử dụng SLEEP và WAKEUP như sau: khi một tiến trình chưa đủ điều kiện vào miền găng, nó gọi SLEEP để tự khóa đến khi có một tiến trình khác gọi WAKEUP để giải phóng cho nó. Một tiến trình gọi WAKEUP khi ra khỏi miền găng để đánh thức một tiến trình đang chờ, tạo cơ hội cho tiến trình này vào miền găng:

```
int busy; // 1 nếu miền găng đang bị chiếm, nếu không là 0
int blocked; // đếm số lượng tiến trình đang bị khóa
while (TRUE) {
if (busy){
blocked = blocked + 1;
sleep();
} else busy = 1;
critical-section ();
busy = 0;
if(blocked){
wakeup(process);
blocked = blocked - 1; }
Noncritical-section (); }
```

Khi sử dụng SLEEP và WAKEUP cần hết sức cẩn thận, nếu không muốn xảy ra tình trạng mâu thuẫn truy xuất trong một vài tình huống đặc biệt như sau: giả sử tiến trình A vào miền găng, và trước khi nó rời khỏi miền găng thì tiến trình B được kích hoạt. Tiến trình B thử vào miền găng nhưng nó nhận thấy A đang ở trong đó, do vậy B tăng giá trị biến blocked và chuẩn bị gọi SLEEP để tự khoá. Tuy nhiên trước khi B có thể thực hiện SLEEP, tiến trình A lại được tái kích hoạt và ra khỏi miền găng. Khi ra khỏi miền găng A nhận thấy có một tiến trình đang chờ (blocked=1) nên gọi WAKEUP và giảm giá trị của blocked. Khi đó tín hiệu WAKEUP sẽ lạc mất do tiến trình B chưa thật sự “ngủ” để nhận tín hiệu đánh thức. Khi tiến trình B được tiếp tục xử lý, nó mới gọi SLEEP và tự khó vĩnh viễn. Vấn đề ghi nhận được là tình trạng lỗi này xảy ra do việc kiểm tra tư cách vào miền găng và việc gọi SLEEP hay WAKEUP là những hành động tách biệ, có thể bị ngắt nửa chừng trong quá trình xử lý, do đó có khi tín hiệu WAKEUP gởi đến một tiến trình chưa bị khóa sẽ lạc mất.

Để tránh những tình huống tương tự, hệ điều hành cung cấp những cơ chế đồng bộ hóa dựa trên ý tưởng của chiến lược “SLEEP and WAKEUP” nhưng được xây dựng bao hàm cả phương tiện kiểm tra điều kiện vào miền găng giúp sử dụng an toàn.

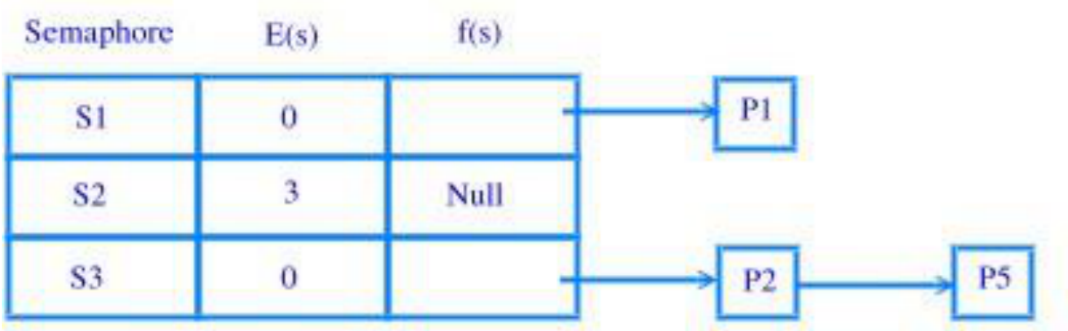
4.4.2.1 Semaphore

Tiếp cận: Được Dijkstra đề xuất vào 1965, một semaphore s là một biến có các thuộc tính sau:

- Một giá trị nguyên dương e(s)
- Một hàng đợi f(s) lưu danh sách các tiến trình đang bị khóa (chờ) trên semaphores
- Chỉ có hai thao tác được định nghĩa trên semaphore

Down(s): giảm giá trị của semaphore s đi 1 đơn vị nếu semaphore có trị e(s) > 0, và tiếp tục xử lý. Ngược lại, nếu e(s) 0, tiến trình phải chờ đến khi e(s) >0.

Up(s): tăng giá trị của semaphore s lên 1 đơn vị. Nếu có một hoặc nhiều tiến trình đang chờ trên semaphore s, bị khóa bởi thao tác Down, thì hệ thống sẽ chọn một trong các tiến trình này để kết thúc thao tác Down và cho tiếp tục xử lý.



***Hình 4.4.2.1-1**. Semaphore s*

Cài đặt: Gọi p là tiến trình thực hiện thao tác Down(s) hay Up(s).

Down(s):

```
e(s) = e(s) - 1;
if e(s) < 0 {
status(P)= blocked;
enter(P,f(s)); }
```

Up(s):

```
e(s) = e(s) + 1;
if s = 0 {
exit(Q,f(s)); //Q là tiến trình đang chờ trên s
status (Q) = ready;
enter(Q,ready-list); }
```

Lưu ý cài đặt này có thể đưa đến một giá trị âm cho semaphore, khi đó trị tuyệt đối của semaphore cho biết số tiến trình đang chờ trên semaphore. Điều quan trọng là các thao

tác này cần thực hiện một cách không bị phân chia, không bị ngắt giữa chừng, có nghĩa là không một tiến trình nào được phép truy xuất đến semaphore nếu tiến trình đang thao tác trên semaphore này chưa kết thúc xử lý hay chuyển sang trạng thái blocked.

Sử dụng: có thể dùng semaphore để giải quyết vấn đề truy xuất độc quyền hay tổ chức phối hợp giữa các tiến trình. Tổ chức truy xuất độc quyền với Semaphores: khái niệm semaphore cho phép bảo đảm nhiều tiến trình cùng truy xuất đến miền găng mà không có sự mâu thuẫn truy xuất. n tiến trình cùng sử dụng một semaphore s, e(s) được khởi gán là 1. Để thực hiện đồng bộ hóa, tất cả các tiến trình cần phải áp dụng cùng cấu trúc chương trình sau đây:

```
while (TRUE) {
```

Down(s)

```
critical-section ();
```

Up(s)

```
Noncritical-section (); }
```

Tổ chức đồng bộ hóa với Semaphores: với semaphore có thể đồng bộ hóa hoạt động của hai tiến trình trong tình huống một tiến trình phải đợi một tiến trình khác hoàn tất thao tác nào đó mới có thể bắt đầu hay tiếp tục xử lý. Hai tiến trình chia sẻ một semaphore s, khởi gán e(s) là 0. Cả hai tiến trình có cấu trúc như sau:

P1:

```
while (TRUE) {
job1();
Up(s); //đánh thức P2 }
```

P2:

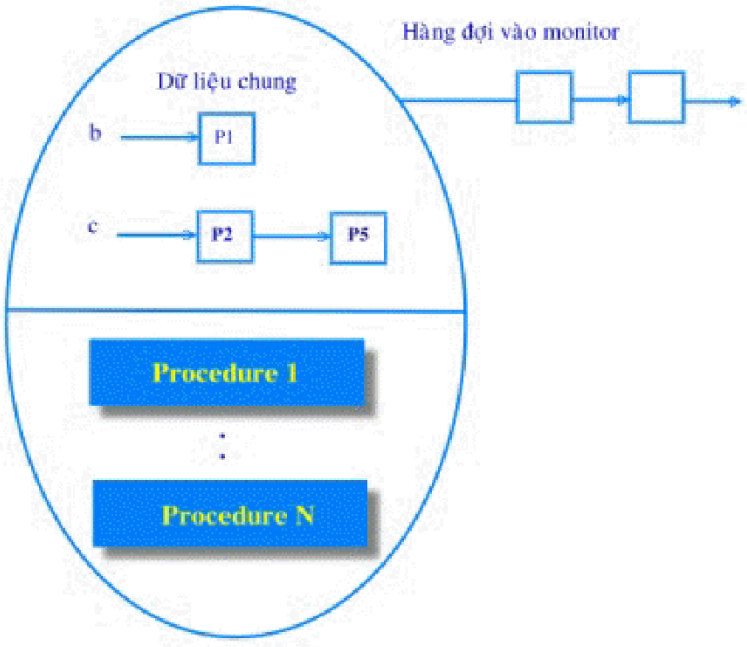
```
while (TRUE) {
Down(s); // chờ P1
job2(); }
```

Việc sử dụng đúng cách semaphore để đồng bộ hóa phụ thuộc hoàn toàn vào lập trình viên và đòi hỏi lập trình viên phải hết sức thận trọng.

4.4.2.2 Monitors

Tiếp cận: Để có thể dễ viết đúng các chương trình đồng bộ hóa hơn, Hoare(1974) và Brinch & Hansen (1975) đã đề nghị một cơ chế cao hơn được cung cấp bởi ngôn ngữ lập trình, là monitor. Monitor là một cấu trúc đặc biệt bao gồm các thủ tục, các biến và cấu trúc dữ liệu có các thuộc tính sau:

- Các biến và cấu trúc dữ liệu bên trong monitor chỉ có thể được thao tác bởi các thủ tục định nghĩa bên trong monitor đó (encapsulation).
- Tại một thời điểm, chỉ có một tiến trình duy nhất được hoạt động bên trong một monitor (mutual exclusive).
- Trong một monitor, có thể định nghĩa các biến điều kiện và hai thao tác kèm theo là Wait và Signal như sau: gọi c là biến điều kiện được định nghĩa trong monitor:
  - Wait(c): chuyển trạng thái tiến trình gọi sang blocked, và đặt tiến trình này vào hàng đợi trên biến điều kiện c.
  - Signal(c): nếu có một tiến trình đang bị khóa trong hàng đợi của c, tái kích hoạt tiến trình đó, và tiến trình gọi sẽ rời khỏi monitor.



***Hình 4.4.2.2-1**.Monitor và các biến điều kiện*

Cài đặt: trình biên dịch chịu trách nhiệm thực hiện việc truy xuất độc quyền đến dữ liệu trong monitor. Để thực hiện điều này, một semaphore nhị phân thường được sử dụng. Mỗi monitor có một hàng đợi toàn cục lưu các tiến trình đang chờ được vào monitor, ngoài ra, mỗi biến điều kiện c cũng gắn với một hàng đợi f(c) và hai thao tác trên đó được định nghĩa như sau:

Wait(c) :

```
status(P)= blocked;
enter(P,f(c));
```

Signal(c) :

```
if (f(c) != NULL){
exit(Q,f(c)); //Q là tiến trình chờ trên c
statusQ) = ready;
enter(Q,ready-list); }
```

Sử dụng: Với mỗi nhóm tài nguyên cần chia sẻ, có thể định nghĩa một monitor trong đó đặc tả tất cả các thao tác trên tài nguyên này với một số điều kiện nào đó:

```
monitor <tên monitor >
```

condition <danh sách các biến điều kiện>; <déclaration de variables>;

```
procedure Action1();
{
}
procedure Actionn();
{
}
end monitor;
```

Các tiến trình muốn sử dụng tài nguyên chung này chỉ có thể thao tác thông qua các thủ tục bên trong monitor được gắn kết với tài nguyên:

```
while (TRUE) {
Noncritical-section ();
<monitor>.Actioni; //critical-section();
Noncritical-section (); }
```

4.4.2.3 Trao đổi thông điệp

Tiếp cận: giải pháp này dựa trên cơ sở trao đổi thông điệp với hai primitive Send và Receive để thực hiện sự đồng bộ hóa:

- Send (destination, message): gởi một thông điệp đến một tiến trình hay gởi vào hộp thư.
- Receive (source,message): nhận một thông điệp thừ một tiến trình hay từ bất kỳ một tiến trình nào, tiến trình gọi sẽ chờ nếu không có thông điệp nào để nhận.

Sử dụng: Có nhiều cách thức để thực hiện việc truy xuất độc quyền bằng cơ chế trao đổi thông điệp. Đây là một mô hình đơn giản: một tiến trình kiểm soát việc sử dụng tài nguyên và nhiều tiến trình khác yêu cầu tài nguyên này. Tiến trình có yêu cầu tài nguyên sẽ gởi một thông điệp đến tiến trình kiểm soát và sau đó chuyển sang trạng thái blocked cho đến khi nhận được một thông điệp chấp nhận cho truy xuất từ tiến trình kiểm soát tài nguyên.Khi sử dụng xong tài nguyên, tiến trình gởi một thông điệp khác đến tiến trình kiểm soát để báo kết thúc truy xuất. Về phần tiến trình kiểm soát , khi nhận được thông điệp yêu cầu tài nguyên, nó sẽ chờ đến khi tài nguyên sẵn sàng để cấp phát thì gởi một thông điệp đến tiến trình đang bị khóa trên tài nguyên đó để đánh thức tiến trình này.

```
while (TRUE) {
Send(process controler, request message);
Receive(process controler, accept message);
critical-section ();
Send(process controler, end message);
Noncritical-section (); }
```

Câu hỏi ôn tập

1. Vì sao giải pháp dùng biến khóa (lock) không đảm bảo độc quyền truy xuất?
2. Giải thích vì sao giải pháp Peterson đảm bảo độc quyền truy xuất cho hai tiến trình.
3. Nhược điểm chung của các giải pháp busy waiting là gì?
4. Trình bày định nghĩa và cách cài đặt các thao tác Down(s), Up(s) của semaphore.
5. Monitor khác semaphore ở điểm nào? Vì sao monitor giúp giảm lỗi lập trình?

Học thêm, bài tập và hướng dẫn giải

Nguyên lí hệ điều hành — Bài tập mở rộng về tiến trình và đồng bộ

Tài liệu tự biên soạn bổ sung cho 11 bài hiện có, không thay thế bài giảng gốc. Phân biệt mã giả với API cụ thể; ví dụ về đồng bộ dùng mô hình lý thuyết.

Mục tiêu học thêm

- Phân biệt đồng thời, song song, tiến trình và tiểu trình
- Phân tích trạng thái tiến trình và lựa chọn cơ chế liên lạc
- Tìm race condition, deadlock và lỗi thứ tự lấy khóa

1. Đồng thời và song song

Đồng thời nghĩa là nhiều công việc có tiến triển trong các khoảng thời gian chồng lấn; trên một lõi, hệ điều hành có thể xen kẽ chúng. Song song là thực thi tại cùng thời điểm trên nhiều đơn vị xử lý. Có nhiều thread không bảo đảm chương trình nhanh hơn: còn phụ thuộc khối lượng công việc, chờ I/O, tranh chấp khóa và chi phí chuyển ngữ cảnh.

Tiến trình có không gian địa chỉ và tài nguyên riêng theo mô hình hệ điều hành. Các thread cùng tiến trình thường chia sẻ không gian địa chỉ nhưng mỗi thread có stack và trạng thái thực thi riêng. Chia sẻ dữ liệu thuận tiện hơn, đồng thời cũng dễ phát sinh tranh chấp.

2. Trạng thái và lập lịch

Một mô hình đơn giản có ready, running và blocked. Ready là đủ điều kiện chạy nhưng chờ CPU; blocked là đang chờ sự kiện như I/O. Khi hết quantum, tiến trình running có thể quay về ready. Hoàn tất I/O thường đưa blocked về ready, không nhất thiết được chạy ngay.

Ví dụ Round Robin: A cần 5 đơn vị CPU, B cần 3, cùng tới thời điểm 0, quantum=2, hàng đợi ban đầu A rồi B, không có I/O và bỏ qua chi phí chuyển ngữ cảnh.

- A chạy 0–2, còn 3.
- B chạy 2–4, còn 1.
- A chạy 4–6, còn 1.
- B chạy 6–7, hoàn tất.
- A chạy 7–8, hoàn tất.

Turnaround của A là 8, B là 7. Waiting time bằng turnaround trừ tổng CPU trong mô hình này: A=3, B=4. Nếu có I/O thì không áp dụng máy móc công thức trừ CPU mà bỏ qua thời gian bị blocked.

3. Lựa chọn IPC

Pipe phù hợp luồng dữ liệu giữa các tiến trình trong tình huống thích hợp trên cùng máy; socket phù hợp liên lạc qua mạng hoặc cùng máy. Shared memory cho phép chia sẻ vùng dữ liệu nhưng cần tự giải quyết đồng bộ và quyền truy cập. Message passing giúp tách vùng nhớ, nhưng phải xem xét kích thước thông điệp, sao chép và việc gửi/nhận có chặn hay không.

Không chọn IPC chỉ vì "nhanh nhất". Cần biết các tiến trình có cùng máy không, có quan hệ cha/con không, dữ liệu lớn bao nhiêu, có bao nhiêu bên gửi/nhận và cần xử lý lỗi bên kia dừng thế nào.

4. Race condition

Phép counter = counter + 1 có thể gồm đọc, tính và ghi, không nên mặc định nguyên tử. Hai thread cùng đọc 0, cùng tính 1 rồi cùng ghi 1 làm mất một lần tăng; kết quả đúng mong đợi là 2.

Mutex bảo vệ vùng đọc-sửa-ghi; mọi đường truy cập dữ liệu chia sẻ cần tuân thủ cùng quy tắc khóa. Không chỉ khóa dòng ghi cuối vì hai thread vẫn có thể tính từ dữ liệu cũ. Cần bảo đảm khóa được giải phóng khi có lỗi.

5. Producer–consumer và thứ tự semaphore

Với buffer có N ô, đặt empty=N, full=0 và mutex=1. wait giảm semaphore hoặc chờ khi không thể giảm; signal tăng và có thể đánh thức bên chờ.

```
Producer:
  wait(empty)
  wait(mutex)
  thêm dữ liệu vào buffer
  signal(mutex)
  signal(full)

Consumer:
  wait(full)
  wait(mutex)
  lấy dữ liệu khỏi buffer
  signal(mutex)
  signal(empty)
```

Không lấy mutex rồi mới chờ empty/full: nếu giữ mutex trong khi buffer đầy/rỗng, bên cần thay đổi buffer có thể không lấy được mutex để giải phóng điều kiện chờ. Với condition variable, thường kiểm tra điều kiện trong vòng while dưới mutex và dùng thao tác chờ phù hợp giải phóng mutex; không dùng if một lần rồi giả định điều kiện luôn còn đúng khi thức dậy.

6. Deadlock và starvation

Deadlock là các bên không thể tiếp tục do chờ tài nguyên nhau. Ví dụ T1 giữ A chờ B, T2 giữ B chờ A. Quy định mọi thread lấy A trước B loại vòng chờ này. Starvation là một bên liên tục không được phục vụ dù hệ thống vẫn tiến triển; không đồng nghĩa với deadlock. Một khóa có mutual exclusion chưa tự bảo đảm fairness.

Bài tập và gợi ý đáp án

1. Dễ: I/O hoàn tất có nghĩa tiến trình chạy ngay không? Không: thông thường chuyển về ready rồi chờ lập lịch.
2. Trung bình: lập bảng Round Robin ở ví dụ và tính thời điểm phản hồi đầu tiên. A=0, B=2; kiểm tra tổng CPU là 8.
3. Khó: producer giữ mutex rồi chờ empty khi buffer đầy. Consumer cần mutex để lấy phần tử nên không giải phóng empty được; đổi thứ tự như mã giả trên. Phân tích thêm lỗi nếu quên signal(mutex).

Tiêu chí và nguồn học tiếp

Vẽ timeline xen kẽ từng thao tác, ghi trạng thái biến và giá trị semaphore; không chỉ kết luận "có lỗi". Với lời giải đồng bộ, chỉ ra vùng tới hạn, điều kiện chờ và bên có thể làm điều kiện thay đổi. Với lập lịch, luôn ghi giả định về thời điểm đến, quantum và chi phí chuyển ngữ cảnh.

https://pages.cs.wisc.edu/~remzi/OSTEP/ ; các chương CPU Scheduling, Threads, Locks, Condition Variables và Semaphores.

Bài 11: Các giải pháp đồng bộ hóa — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: dùng lock nhưng vẫn treo vì thứ tự lấy nhiều khóa không nhất quán. Quy định cùng một thứ tự lấy khóa và giải phóng ở mọi đường thoát. Semaphore đếm tài nguyên; mutex bảo vệ vùng truy cập theo hợp đồng API, không dùng hai khái niệm như đồng nghĩa tuyệt đối.

Tự luyện: mô hình hàng đợi hữu hạn producer/consumer: giới hạn sức chứa, số phần tử và chỗ trống. **Hướng giải:** kiểm tra bất biến 0 ≤ size ≤ capacity, không giữ khóa lâu trong I/O nếu không cần. Với condition variable, kiểm tra lại điều kiện sau khi thức dậy. Muốn tìm ứng dụng nên đóng trên máy lag phải có dữ liệu thực; không đoán tên chương trình từ tài liệu. Bộ nhớ ảo mở rộng mô hình không gian địa chỉ nhưng paging ra đĩa không làm RAM vật lý nhanh hơn.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/